

Laboratório Especial — Trajetos de Aeronaves

AUTHOR

Otavio Ribeiro Mello

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.1.4      ✓ readr      2.1.5
✓ forcats    1.0.0      ✓ stringr    1.5.1
✓ ggplot2    3.5.2      ✓ tibble     3.2.1
✓ lubridate  1.9.4      ✓ tidyr      1.3.1
✓ purrr      1.0.4

— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(leaflet)
library(lubridate)
library(scales)
```

Anexando pacote: 'scales'

O seguinte objeto é mascarado por 'package:purrr':

```
discard
```

O seguinte objeto é mascarado por 'package:readr':

```
col_factor
```

a.Receba um valor de TAIL_NUMBER (por exemplo, N431WN);

b.Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip);

c.Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima);

d.O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do vôo como espessura da linha que conecta os aeroportos envolvidos no trajeto);

```
#' Analisa o trajeto histórico de uma aeronave ao longo do ano
#'
#' Dado o código de cauda (TAIL_NUMBER) de uma aeronave, a função varre o
#' arquivo de voos EM PARTES (pois ele pode ser muito grande), mantém
#' apenas os voos daquela aeronave, ordena cronologicamente e devolve
#' (1) a tabela tidy correspondente e (2) um mapa leaflet com o trajeto.
```

```

#'
#' @param tail_number Código identificador da aeronave (ex.: "N431WN").
#' @param arquivo Caminho completo para o arquivo de voos (.csv ou
#'   .csv.zip), ex.: file.path("dados", "flights.csv.zip").
#' @param arquivo_aeroportos Caminho para o arquivo airports.csv, usado
#'   para obter as coordenadas exatas dos aeroportos. Se NULL (padrão), a
#'   função procura um arquivo "airports.csv" na mesma pasta de `arquivo`.
#'
#' @return Lista nomeada com dois elementos:
#'   - tabela: tibble com todos os voos da aeronave, ordenados por data/hora;
#'   - mapa: objeto leaflet com o trajeto da aeronave (ou NULL se o arquivo
#'     de aeroportos não for encontrado).
analisa_aeronave <- function(tail_number, arquivo, arquivo_aeroportos = NULL) {

  # --- 1. Especificação explícita das colunas -----
  # Fixar os tipos de coluna evita dois problemas comuns na leitura em
  # partes: (a) o readr adivinhar tipos diferentes em chunks diferentes, e
  # (b) colunas de horário como "0005" perderem o zero à esquerda ao virar
  # número (nós reconstituímos esse zero manualmente mais adiante).
  especificacao_colunas <- readr::cols(
    YEAR           = readr::col_integer(),
    MONTH          = readr::col_integer(),
    DAY            = readr::col_integer(),
    DAY_OF_WEEK    = readr::col_integer(),
    AIRLINE        = readr::col_character(),
    FLIGHT_NUMBER  = readr::col_integer(),
    TAIL_NUMBER    = readr::col_character(),
    ORIGIN_AIRPORT = readr::col_character(),
    DESTINATION_AIRPORT = readr::col_character(),
    SCHEDULED_DEPARTURE = readr::col_integer(),
    DEPARTURE_TIME = readr::col_integer(),
    DEPARTURE_DELAY = readr::col_double(),
    TAXI_OUT       = readr::col_double(),
    WHEELS_OFF     = readr::col_integer(),
    SCHEDULED_TIME = readr::col_double(),
    ELAPSED_TIME   = readr::col_double(),
    AIR_TIME       = readr::col_double(),
    DISTANCE       = readr::col_double(),
    WHEELS_ON      = readr::col_integer(),
    TAXI_IN        = readr::col_double(),
    SCHEDULED_ARRIVAL = readr::col_integer(),
    ARRIVAL_TIME   = readr::col_integer(),
    ARRIVAL_DELAY  = readr::col_double(),
    DIVERTED       = readr::col_integer(),
    CANCELLED      = readr::col_integer(),
    CANCELLATION_REASON = readr::col_character(),
    AIR_SYSTEM_DELAY = readr::col_double(),
    SECURITY_DELAY  = readr::col_double(),
    AIRLINE_DELAY  = readr::col_double(),
    LATE_AIRCRAFT_DELAY = readr::col_double(),
    WEATHER_DELAY  = readr::col_double()
  )

  # --- 2. Se o arquivo vier compactado (.zip), extrai antes de ler -----

```

```

# A leitura em partes (chunked) nem sempre lida bem com conexões .zip,
# então extraímos o CSV para uma pasta temporária primeiro.
if (tolower(tools::file_ext(arquivo)) == "zip") {
  pasta_temporaria <- tempdir()
  nome_csv_extraido <- utils::unzip(arquivo, exdir = pasta_temporaria, list = TRUE)$Name[1]
  utils::unzip(arquivo, exdir = pasta_temporaria)
  arquivo <- file.path(pasta_temporaria, nome_csv_extraido)
}

# --- 3. Leitura do arquivo de voos EM PARTES (chunks) -----
# Em vez de carregar o arquivo inteiro em memória, lemos em blocos de
# `tamanho_do_bloco` linhas e, a cada bloco, mantemos só as linhas da
# aeronave de interesse (callback). Isso é essencial para um arquivo do
# tamanho do flights.csv.zip completo.
filtra_bloco_pela_aeronave <- function(bloco, posicao) {
  dplyr::filter(bloco, TAIL_NUMBER == tail_number)
}

tamanho_do_bloco <- 50000

voos_da_aeronave <- readr::read_csv_chunked(
  file      = arquivo,
  callback  = readr::DataFrameCallback$new(filtra_bloco_pela_aeronave),
  chunk_size = tamanho_do_bloco,
  col_types = especificacao_colunas
)

if (nrow(voos_da_aeronave) == 0) {
  stop(sprintf("Nenhum voo encontrado para a aeronave '%s' em '%s'.", tail_number, arquivo))
}

# --- 4. Data/hora de partida, para permitir a ordenação cronológica -----
# SCHEDULED_DEPARTURE vem no formato HHMM (ex.: 5 representa 00:05).
# sprintf("%04d", ...) reconstitui o zero à esquerda antes de separar
# hora e minuto.
voos_da_aeronave <- voos_da_aeronave |>
  dplyr::mutate(
    horario_partida_hhmm = sprintf("%04d", SCHEDULED_DEPARTURE),
    hora_partida         = as.integer(substr(horario_partida_hhmm, 1, 2)),
    minuto_partida       = as.integer(substr(horario_partida_hhmm, 3, 4)),
    data_hora_partida    = lubridate::make_datetime(
      year = YEAR, month = MONTH, day = DAY,
      hour = hora_partida, min = minuto_partida
    )
  ) |>
  dplyr::arrange(data_hora_partida) |>
  dplyr::select(-horario_partida_hhmm, -hora_partida, -minuto_partida)

# --- 5. Tabela final (tidy), com número sequencial do trecho -----
# numero_sequencial reflete a ordem cronológica dos voos e é usado tanto
# na tabela quanto para colorir o mapa por ordem no tempo.
# velocidade_media_mph é a estatística usada para decorar o mapa
# (espessura da linha): distância voada dividida pelo tempo de voo.
tabela_aeronave <- voos_da_aeronave |>

```

```

dplyr::mutate(
  numero_sequencial = dplyr::row_number(),
  velocidade_media_mph = dplyr::if_else(
    !is.na(AIR_TIME) & AIR_TIME > 0,
    DISTANCE / (AIR_TIME / 60),
    NA_real_
  )
)

# --- 6. Localiza o arquivo de aeroportos -----
if (is.null(arquivo_aeroportos)) {
  arquivo_aeroportos <- file.path(dirname(arquivo), "airports.csv")
}

# Se não houver arquivo de aeroportos, devolvemos a tabela sem
# coordenadas e sem mapa, em vez de interromper a função com um erro.
if (!file.exists(arquivo_aeroportos)) {
  warning(sprintf(
    paste(
      "Arquivo de aeroportos nao encontrado em '%s'.",
      "A tabela foi retornada sem coordenadas e o mapa nao foi gerado.",
      "Informe o argumento 'arquivo_aeroportos' ou coloque 'airports.csv'",
      "na mesma pasta de 'arquivo' para obter o mapa."
    ),
    arquivo_aeroportos
  ))
  return(list(tabela = tabela_aeronave, mapa = NULL))
}

# --- 7. Coordenadas dos aeroportos e junção com a tabela -----
aeroportos <- readr::read_csv(arquivo_aeroportos, show_col_types = FALSE) |>
dplyr::select(
  codigo_iata = IATA_CODE,
  nome_aeroporto = AIRPORT,
  cidade = CITY,
  latitude = LATITUDE,
  longitude = LONGITUDE
)

tabela_aeronave <- tabela_aeronave |>
dplyr::left_join(aeroportos, by = c("ORIGIN_AIRPORT" = "codigo_iata")) |>
dplyr::rename(
  aeroporto_origem = nome_aeroporto,
  cidade_origem = cidade,
  latitude_origem = latitude,
  longitude_origem = longitude
) |>
dplyr::left_join(aeroportos, by = c("DESTINATION_AIRPORT" = "codigo_iata")) |>
dplyr::rename(
  aeroporto_destino = nome_aeroporto,
  cidade_destino = cidade,
  latitude_destino = latitude,
  longitude_destino = longitude
)

```

```
# --- 8. Construção do mapa (leaflet) -----
# Cor do trecho = ordem cronológica (paleta viridis, do início ao fim do
# ano); espessura do trecho = velocidade média do voo.
paleta_ordem_cronologica <- leaflet::colorNumeric(
  palette = "viridis",
  domain = tabela_aeronave$numero_sequencial
)

intervalo_velocidade <- range(tabela_aeronave$velocidade_media_mph, na.rm = TRUE)
usar_espessura_fixa <- !is.finite(intervalo_velocidade[1]) || diff(intervalo_velocidade) ==

mapa_aeronave <- leaflet::leaflet(tabela_aeronave) |>
  leaflet::addProviderTiles(leaflet::providers$CartoDB.Positron) |>
  leaflet::addCircleMarkers(
    lng = ~longitude_origem, lat = ~latitude_origem,
    radius = 4, color = "#333333", fillOpacity = 0.8,
    label = ~paste(ORIGIN_AIRPORT, "-", aeroporto_origem)
  ) |>
  leaflet::addCircleMarkers(
    lng = ~longitude_destino, lat = ~latitude_destino,
    radius = 4, color = "#333333", fillOpacity = 0.8,
    label = ~paste(DESTINATION_AIRPORT, "-", aeroporto_destino)
  )

# Cada trecho vira uma polilinha própria, para poder variar espessura e
# cor trecho a trecho (refletindo velocidade e ordem no tempo).
for (indice_trecho in seq_len(nrow(tabela_aeronave))) {
  trecho <- tabela_aeronave[indice_trecho, ]

  espessura_linha <- if (usar_espessura_fixa || is.na(trecho$velocidade_media_mph)) {
    4
  } else {
    scales::rescale(trecho$velocidade_media_mph, to = c(2, 8), from = intervalo_velocidade)
  }

  texto_popup <- sprintf(
    paste0(
      "<b>Trecho %d</b><br>",
      "%s &rarr; %s<br>",
      "Data: %s<br>",
      "Velocidade media: %s mph<br>",
      "Atraso na chegada: %s min"
    ),
    trecho$numero_sequencial,
    trecho$ORIGIN_AIRPORT, trecho$DESTINATION_AIRPORT,
    as.character(as.Date(trecho$data_hora_partida)),
    ifelse(is.na(trecho$velocidade_media_mph), "N/D", round(trecho$velocidade_media_mph, 0)),
    ifelse(is.na(trecho$ARRIVAL_DELAY), "N/D", trecho$ARRIVAL_DELAY)
  )

  mapa_aeronave <- mapa_aeronave |>
    leaflet::addPolygons(
      lng = c(trecho$longitude_origem, trecho$longitude_destino),
```

```

    lat      = c(trecho$latitude_origem, trecho$latitude_destino),
    weight    = espessura_linha,
    color     = paleta_ordem_cronologica(trecho$numero_sequencial),
    opacity   = 0.85,
    popup     = texto_popup
  )
}

mapa_aeronave <- mapa_aeronave |>
  leaflet::addLegend(
    position = "bottomright",
    pal      = paleta_ordem_cronologica,
    values   = tabela_aeronave$numero_sequencial,
    title     = "Ordem do trecho<br>(cronologica)"
  )

# --- 9. Retorno -----
list(
  tabela = tabela_aeronave,
  mapa   = mapa_aeronave
)
}

```

```

pasta_dados <- "C:/Users/Usuario/Downloads"

file.exists(file.path(pasta_dados, "flights.csv"))

```

[1] TRUE

```
file.exists(file.path(pasta_dados, "airports.csv"))
```

[1] TRUE

```

# N407AS aparece duas vezes na amostra (ANC -> SEA -> LAS), então serve
# para checar se a ordenação cronológica e o mapa estão funcionando.
resultado_amostra <- analisa_aeronave(
  tail_number = "N407AS",
  arquivo     = file.path(pasta_dados, "flights.csv")
  # arquivo_aeroportos não precisa ser informado: a função já encontra
  # "airports.csv" sozinha, pela mesma pasta de "arquivo" (pasta_dados).
)

```

Warning in validateCoords(lng, lat, funcName): Data contains 109 rows with either missing or invalid lat/lon values and will be ignored
Warning in validateCoords(lng, lat, funcName): Data contains 109 rows with either missing or invalid lat/lon values and will be ignored

```

resultado_amostra$tabela |>
  dplyr::select(
    numero_sequencial, data_hora_partida,

```

```
ORIGIN_AIRPORT, DESTINATION_AIRPORT,  
aeroporto_origem, aeroporto_destino,  
velocidade_media_mph, ARRIVAL_DELAY  
)
```

```
# A tibble: 1,234 × 8  
  numero_sequencial data_hora_partida ORIGIN_AIRPORT DESTINATION_AIRPORT  
      <int> <dtm>          <chr>          <chr>  
1         1 2015-01-01 00:05:00 ANC             SEA  
2         2 2015-01-01 06:05:00 SEA             LAS  
3         3 2015-01-01 09:10:00 LAS             SEA  
4         4 2015-01-01 12:35:00 SEA             ORD  
5         5 2015-01-01 19:30:00 ORD             SEA  
6         6 2015-01-02 06:00:00 SEA             ANC  
7         7 2015-01-02 10:35:00 ANC             SEA  
8         8 2015-01-02 15:55:00 SEA             EWR  
9         9 2015-01-03 07:00:00 EWR             SEA  
10        10 2015-01-03 11:10:00 SEA             LAX  
# i 1,224 more rows  
# i 4 more variables: aeroporto_origem <chr>, aeroporto_destino <chr>,  
#   velocidade_media_mph <dbl>, ARRIVAL_DELAY <dbl>
```

```
resultado_amostra$mapa
```

